



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

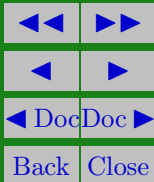
P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

JKUAT SODEL
©2017





ICS 2104 Object-Oriented Programming I

This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 6

Storage classes

Learning outcomes

Upon completing this topic you should be able to:

- Differentiate between scope and lifetime of an object
- Use register, auto, static and extern keywords
- List limitations of register variables

6.1. Introduction

In C++, just like in C, variables and functions have scope and lifetime.

The scope (also referred to as visibility) of a variable or a function is a portion of a program in which it is accessible while



ICS 2104 Object-Oriented Programming I

the lifetime of a variable or a function is the duration of time that it exist during the execution of the program. Information about scope and lifetime of variables and functions is provided by preceding their declaration with the following storage class specifiers:

- auto
- register
- static
- extern

Auto, register, static and extern are keywords.



6.2. Auto storage specifier

All local variables are by default auto. That means they need not be declared with the keyword auto. Since

```
{  
    auto int x;  
} is equivalent to  
{  
    int x;  
}
```

All local variables are created and destroyed automatically.

The word auto is derived from their automatic creation and destruction.

All local variables are created when the block within which they have declared is entered and destroyed when the block is



ICS 2104 Object-Oriented Programming I

exited. Parameters behavior like local variables and therefore they are created and destroyed automatically.

Only local variables are associated with the auto storage class specifier.

Local variables and parameters have local lifetime and they are visible only in the block of code they have been declared.

6.3. The register storage class specifier

All local variables and parameters are stored in computer memory. However, a compiler can be requested to allocate memory to a local variable and a parameter in the system' register. Examples:

```
register int product=1;  
for (register int i=1; i<10; i++)
```



ICS 2104 Object-Oriented Programming I

product*=i;

It is possible that the register is not available. In that case the variable is treated like a local variable. Register variables are quickly accessed and therefore the program runs faster. Since registers are limited only a few variables that are frequently accessed need to be stored in the registers. Register variables have the same lifetime and visibility as auto variables.

Some restriction on register variables

a). Only local variables and parameters can be declared with register storage class specifier

6.4. The static storage class specifier

Static variables and functions are declared using static specifier, for example:

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

```
static int count=0;
```

Static variables have global lifetime. This means that they are created when the program start executing and remain in existence during the entire period of execution. But they are visible only within the block in which they have been declared. Unlike auto variables, static variables are not destroyed after the block in which they are declared has been exited. This means that they keep their values when the block in which they are declared is exited. Variables that have global lifetime are automatically initialization with zero.

Static keyword can also precede declaration of global variables. A global variable is a variables that has been declared outside all the functions in a program.

In C++, functions can not be declared within other func-



ICS 2104 Object-Oriented Programming I

tions. Function can not be declared at internal(local) level. All functions are declared at external(global) level and therefore their declaration can be preceded with static keyword. A static function is only accessible within source file that it has been declared. Functions in the same source file can call static functions but not functions in other source files. Thereafter different source files can have static functions that have the same name.

6.5. The extern storage class specifier

local variables that have been declared with extern keyword are references to variables with the same name and data type that have been declared later in same file or in a different file.



Revision Questions

EXERCISE 1. Differentiate scope and lifetime of a variable

Example . Explain why scope and lifetime of variables and functions are important

Solution: Scope and lifetime of variables enables a program to have the same functions and variables without risking name collision

□

EXERCISE 2. Explain why a function can not be declared with auto and register keyword

EXERCISE 3. Differentiate static and extern storage classes

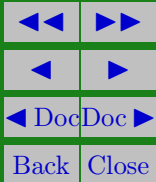
EXERCISE 4. Explain the difference between external storage



JKUAT SODeL
©2017

ICS 2104 Object-Oriented Programming I

and internal storage.



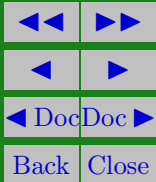


References and Additional Readings

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,
- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.).



1. .





Solutions to Exercises

Exercise 1. Scope of a variable is the portion of the program within which a variable is accessible while life-time of a variable is the duration of time that a variable exist in the program

Exercise 1